

MCP in Claude Code and Hermes Agent

What the Model Context Protocol connects, and who talks to whom · Prepared 2026-10-07

1. Claude Code uses MCP servers

Claude Code is an MCP client. MCP, the Model Context Protocol, is an open standard for exposing tools, resources, and prompts to a model over a defined interface. Claude Code connects to MCP servers and presents their tools to the model alongside its built-in ones.

It is visible in a running session. GitHub tools are named `mcp__github__*`, and the session-management tools are `mcp__claude-code-remote__*`. The prefix is how Claude Code names tools that come from an MCP server rather than from its own code.

How it is set up

- **Adding a server.** `claude mcp add <name> <command>` for a local server that speaks over stdio, or `claude mcp add --transport http <name> <url>` for a remote one. Servers can also be declared in a `.mcp.json` file at the project root so the whole team gets them, or in user settings for every project.
- **Transports.** Local servers run as a subprocess and talk over stdin and stdout. Remote servers use HTTP or SSE, often with OAuth.
- **Scopes.** Local to you, shared through the project's `.mcp.json`, or global in your user config.
- **At runtime.** Claude Code starts or connects to each server, asks it what tools it offers, and lists them to the model. When the model calls one, Claude Code forwards the call to the server and returns the result. Permission prompts apply the same way they do for built-in tools.

Claude Code can also run as an MCP server itself with `claude mcp serve`, which lets another MCP client, such as Claude Desktop, use Claude Code's file and shell tools.

In the terms of the Hermes skill, Colang, and Pipecat comparison, MCP sits on the Hermes-skill side of the line. An MCP server extends what the model can call, and the model decides whether to call it. It adds capabilities, not enforcement.

2. Claude Code does not call an MCP server at Anthropic

Two separate connections are in play, and only one of them goes to Anthropic:

- **The model call.** Claude Code sends the conversation to the Claude API at Anthropic to get the model's next response. That is a plain HTTPS API call, not MCP.
- **The MCP connections.** These go to whichever servers you configured. A local server is a subprocess on your machine talking over stdin and stdout. A remote server is an HTTP endpoint run by whoever operates it, such as GitHub for the GitHub MCP server. Anthropic is not in that path at all unless you happen to configure an Anthropic-hosted server.

The loop

- 1 Claude Code asks each MCP server what tools it has.

- 2 Claude Code tells the model about those tools in the API request to Anthropic.
- 3 The model replies with a tool call.
- 4 Claude Code forwards that call to the MCP server that owns it.
- 5 The result goes back into the next API request.

```

      model call (HTTPS, not MCP)
Claude Code <-----> Claude API (Anthropic)
      |
      | MCP (stdio subprocess, or HTTP/SSE)
+-----> GitHub MCP server      (run by GitHub)
+-----> local filesystem server (subprocess on your machine)
+-----> any other configured server

```

The MCP server never talks to Anthropic, and Anthropic never talks to the MCP server. Claude Code is the broker in the middle. In a cloud session, for example, the GitHub tools come from a GitHub-hosted MCP server, and the session-management tools come from an MCP server that Anthropic's cloud environment runs for Claude Code. Both are MCP servers, but what makes them MCP is the protocol, not who hosts them.

3. Hermes Agent is also an MCP client, and an MCP server

Checked against the hermes-agent source:

As a client

- `tools/mcp_tool.py` and `tools/mcp_tool_discovery.py` connect to configured servers, discover their tools, and register them as `mcp_*` tools for the model.
- The config example describes stdio servers that spawn a subprocess and remote ones over HTTP.
- An `optional-mcps` directory holds ready-made configs for dozens of services such as Asana, Atlassian, Airtable, Algolia, Amplitude, and AWS Knowledge.
- There is a separate result-size spillover threshold for MCP tool results (50,000 characters versus the 100,000 default) because MCP servers routinely return large un-paginated payloads.

As a server

`mcp_serve.py` runs `hermes mcp serve`, a stdio MCP server that exposes Hermes's messaging conversations as tools so that Claude Code, Cursor, or Codex can list conversations, read history, send messages, poll live events, and manage approvals through it. The client-side config is one line:

```
{"mcpServers": {"hermes": {"command": "hermes", "args": ["mcp", "serve"]}}}
```

4. Claude Code and Hermes Agent as MCP participants, side by side

	Claude Code	Hermes Agent
Acts as MCP client	Yes	Yes
Acts as MCP server	Yes, <code>claude mcp serve</code>	Yes, <code>hermes mcp serve</code>

How servers are added	<code>c laude mcp add</code> , a project <code>.mcp.json</code> , or user settings	<code>mcp_servers</code> block in <code>config.yaml</code> , plus ready-made entries under <code>optional-mcps/</code>
Transports supported	stdio subprocess, HTTP, SSE	stdio subprocess, HTTP
Tool naming in the model's view	<code>mcp__<server>__<tool></code>	<code>mcp_<server>_<tool></code>
Tool discovery	At session start, servers are queried for their tool list; deferred tools can be loaded on demand via ToolSearch	At startup during the agent build, via <code>mcp_tool_discovery.py</code> ; slow or unreachable servers delay the build up to a configurable timeout
Permission handling	Per-call permission prompts, same system as built-in tools; allowlists in settings	Plugin guard context and approval flow; dangerous-command guards live at the tool layer
Large results	Returned to the model; some tools paginate	Spilled to disk above 50,000 characters, tighter than the 100,000 default for other tools
OAuth for remote servers	Built in for HTTP/SSE servers	<code>mcp_oauth_manager.py</code>
What its own server exposes	File and shell tools, so another client can drive Claude Code	Messaging conversations: list, read history, send, poll events, manage approvals
Model backend	Claude API at Anthropic over HTTPS, separate from MCP	Configurable providers (Anthropic, OpenAI-compatible, OpenRouter, others) over HTTPS, separate from MCP
Bundled server catalog	No bundled catalog; community and vendor servers are added by the user	Yes, <code>optional-mcps/</code> with dozens of pre-written service configs

The structural picture is identical: each agent is a broker between a model API on one side and a set of MCP servers on the other, and each can also stand on the server side for the other. The differences are in packaging: how servers are declared, how results are managed, and what each one's own server exposes.

5. Summary

- Both Claude Code and Hermes Agent are MCP clients: each connects to configured MCP servers to gain tools.
- Both can also be MCP servers: Claude Code via `c laude mcp serve`, Hermes via `hermes mcp serve`, so either can expose itself for the other to call.
- The model call to Anthropic is a separate HTTPS API connection. MCP servers are reached by the agent directly, wherever they are hosted, and never through Anthropic.
- MCP adds capabilities the model may choose to call. It does not add enforcement.